LexAI – AI-Powered Legal Assistant & Contract Intelligence Platform

MCA Project Presentation

Chhatrapati Shivaji Maharaj University

Academic Year: 2026–27

Submitted By:

Rohan Bongane, Harsh Jha, Aishwariya Bhunia, Nitu Sharma

Introduction

- The legal domain is filled with dense terminology and lengthy documents.
- Contracts are obfuscated by specialized jargon, making it difficult for non-lawyers to understand their rights and hidden risks.
- Professional legal counsel is expensive, leading many to sign contracts blindly.
- LexAI aims to democratize basic legal comprehension using AI.
- It acts as a preliminary informational assistant, not a

Problem Statement

- Time-Consuming Manual Review: Reading 50+ page contracts takes immense cognitive effort and time.
- Keyword Search Limitations: Standard 'Ctrl+F' fails if you don't guess the exact legal synonym.
- Privacy Risks of Public AI: Uploading confidential corporate contracts to public chatbots poses severe data leak risks.
- AI Hallucination: Generic AI models often hallucinate standard industry practices instead of reading your actual contract.

Aim and Objectives

- Aim: Build a secure, web-based platform that uses Retrieval-Augmented Generation (RAG) to demystify complex legal documents.
- Objective 1: Securely extract text from PDF and DOCX files locally.
- Objective 2: Implement strict data isolation so users only access their own files.
- Objective 3: Use local vector embeddings (ChromaDB) for semantic search.

Existing System vs Proposed System

- Existing System (Manual / Ctrl+F): Misses semantic meaning. Very slow. Error-prone.
- Existing System (Public ChatGPT): High privacy risk. Often hallucinates legal precedent. Data is used for training.
- Proposed System (LexAI):
- - Semantic Vector Search understands context and meaning.
- - Strict RAG architecture grounds answers in verified text.
- - Local file and vector storage ensures privacy and data isolation.

Proposed System and Key Features

- Document Processing: Secure upload and raw text extraction from PDF/DOCX.
- Automated Summaries: AI generates executive summaries and highlights key clauses.
- Document-Specific Chat: A conversational interface restricted to the user's uploaded contract.
- General Legal Knowledge Base: A separate, admin-curated repository for general legal queries.
- User Isolation: Database-level filtering ensures privacy.

Technology Stack

- Frontend: HTML5, CSS3, Bootstrap 5, JavaScript (DOMPurify, marked.js).
- Backend Framework: Python and Flask.
- Relational Database: MySQL with Flask-SQLAlchemy (User Auth, Metadata).
- Document Parsing: PyMuPDF (PDFs) and python-docx (Word documents).
- Embeddings: SentenceTransformers (all-MiniLM-L6-v2).
- Vector Database: ChromaDB (Local semantic storage).

System Architecture

- User Browser ➔ Flask Web Server ➔ (Authentication & Route Protection)
- Flask ➔ PyMuPDF / python-docx ➔ (Extract Text)
- Flask ➔ SentenceTransformers ➔ (Generate Vectors)
- Vectors ➔ ChromaDB ➔ (Store and Retrieve Semantics)
- Flask ➔ MySQL ➔ (Store Metadata and Chat History)
- Flask ➔ Gemini API ➔ (Generate Final Answer)

System Workflow

- 1. User Registration/Login: Bcrypt password hashing secures the session.
- 2. Upload: System validates file extensions and MIME types.
- 3. Chunking: Text is divided into 500-word chunks with overlapping windows to preserve context.
- 4. Indexing: Chunks are embedded and stored in ChromaDB with 'user_id' and 'doc_id' tags.
- 5. Querying: User asks a question in the chat interface.
- 6. Retrieval: System retrieves top-k chunks mathematically

AI and RAG Working

- RAG = Retrieval-Augmented Generation.
- Why RAG? Standard LLMs have token limits and hallucinate. RAG solves this.
- Step A (Retrieval): The system searches the local ChromaDB for factual text segments matching the query.
- Step B (Augmentation): The retrieved text is injected into a strict system prompt.
- Step C (Generation): The LLM is instructed: 'Answer ONLY using the provided text.'

Database and Security

- MySQL Authentication: Users are managed via Flask-Login. Passwords are salted and hashed using Bcrypt.
- Document Isolation: Uploaded files are stored securely on the local filesystem.
- Vector Filtering: Every ChromaDB query strictly filters by `user_id == current_user.id`.
- Data Separation: The General Legal Knowledge Base uses a completely separate ChromaDB collection from private documents.

Testing and Validation

- Framework: System reliability is verified using the Pytest framework.
- Authentication Tests: Validated user registration, secure login routes, and session protection.
- Upload Validation: Verified that the system rejects unsupported file types (e.g., .exe).
- Database Isolation Tests: Confirmed that users cannot access documents or vector data belonging to other accounts.
- Fallback Testing: Verified that the local Python parser takes over

Limitations and Future Scope

- Limitation 1: No OCR. The system currently requires documents with embedded text layers. It cannot read scanned images.
- Limitation 2: No Multilingual Support. Processing is currently optimized for English legal text.
- Limitation 3: AI Inaccuracy. While RAG minimizes hallucination, no AI is 100% flawless.
- Future Scope: Integrate Tesseract OCR for scanned PDFs.
- Future Scope: Implement translation pipelines for regional language support.

Conclusion

- LexAI successfully demonstrates a secure, functional prototype for AI-assisted contract intelligence.
- By combining local vector indexing with RAG, we bridged the gap between complex legalese and accessible comprehension.
- Strict database-level isolation ensures user privacy is maintained.
- The deterministic fallback parser guarantees baseline resilience against API outages.
- LexAI serves as a powerful informational assistant, empowering

Thank You

Questions & Answers

LexAI – AI-Powered Legal Assistant & Contract Intelligence Platform